

PERFORMANCE ANALYSIS OF AI ENABLED HEALTHCARE CHATBOT

A project report submitted in partial fulfillment for the award of the degree of

BACHELOR OF TECHNOLOGY IN COMPUTER SCIENCE AND ENGINEERING (DATA SCIENCE)

Submitted By

**GUDE REDDY LEELA
LAVANYA SANGARAJU
MANEESH KUMAR
AKKILI GEYA SREE**

**22BFA32036
22BFA32060
22BFA32001
22BFA32004**

Under the guidance of

M. Sai Kumar, M. Tech.,

Assistant Professor



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING (DATA SCIENCE)

**SRI VENKATESWARA COLLEGE OF ENGINEERING
(AUTONOMOUS)**

**(Approved by AICTE, New Delhi & Permanently Affiliated to JNTUA, Ananthapuramu
Accredited by NBA, New Delhi & NAAC with 'A' grade)
Karakambadi Road, TIRUPATI – 517507**

2022 – 2026

SRI VENKATESWARA COLLEGE OF ENGINEERING

(AUTONOMOUS)

**(Approved by AICTE, New Delhi & Permanently Affiliated to JNTUA, Ananthapuramu
Accredited by NBA, New Delhi & NAAC with 'A' Grade)**

Karakambadi Road, TIRUPATI – 517507.

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING (DATA SCIENCE)



CERTIFICATE

This is to certify that the project work entitled, **“Performance analysis of AI enabled healthcare Chatbot”** is a bonafide record of the project work done and submitted by

Gude Reddy Leela	22BFA32036
Maneesh Kumar	22BFA32001
Lavanya Sangaraju	22BFA32060
Akkili Geya Sree	22BFA32004

under my guidance and supervision for the partial fulfillment of the requirements for the award of B.Tech degree in **COMPUTER SCIENCE AND ENGINEERING(DATA SCIENCE)**. This project is the result of our own effort and that it has not been submitted to any other University or Institution for the award of any degree or diploma other than specified above

GUIDE

HEAD OF THE DEPARTMENT

External Viva-Voce Exam held on _____

INTERNAL EXAMINER

EXTERNAL EXAMINER

DECLARATION

We hereby declare that the project entitled “**Performance analysis of AI enabled healthcare Chatbot**”, done by us under the esteemed guidance of **Mr. M. Sai Kumar, M. Tech.**, and is submitted in partial fulfillment of the requirements for the award of the Degree of **Bachelor of Technology in Computer Science and Engineering (DATA SCIENCE)**.

Date:-

Place:-

Gude Reddy Leela (22BFA32036)

Lavanya Sangaraju (22BFA32060)

Maneesh Kumar (22BFA32001)

Akkili Geya Sree (22BFA32004)

ACKNOWLEDGEMENT

We are thankful to our guide **Mr. M. Sai Kumar, M. Tech., Asst professor.** for his valuable guidance and encouragement. His helping attitude and suggestions have helped us in the successful completion of the project.

We would like to express our gratefulness and sincere thanks to **Dr.Keerthipati Kumar, Head, Dept of CSE (DATA SCIENCE),** for his kind help and encouragement during the course of our study and in the successful completion of the project work.

We have great pleasure in expressing our hearty thanks to our beloved Principal **Prof.Vijaya Gunturu,** for spending his valuable time with us to complete this project.

Successful completion of any project cannot be done without proper support and encouragement. We sincerely thank to the **Management** for providing all the necessary facilities during the course of study.

We would like to thank our parents and friends, who have the greatest contributions in all our achievements, for the great care and blessings in making us successful in all our endeavors.

Gude Reddy Leela	22BFA32036
Lavanya Sangaraju	22BFA32060
Maneesh Kumar	22BFA32001
Akkili Geya Sree	22BFA32004

ABSTRACT

The rapid advancement of Artificial Intelligence has significantly transformed the healthcare sector by enabling intelligent and real-time assistance systems. This project presents the design and development of an AI-Powered Healthcare Chat Assistant, a full-stack web application that provides medical assistance, emergency detection, and healthcare services through an interactive chatbot interface.

The system is built using Java 17 and Spring Boot 3.2 for the backend, ensuring robust and secure server-side operations. React 18 with Vite and Tailwind CSS is used for the frontend to deliver a modern and responsive user interface. The application integrates Ollama (gemma:2b) as the primary AI model for generating responses locally, with OpenAI GPT-4 as a fallback to ensure reliability.

The system supports multi-role access (Patient, Doctor, Admin) using JWT-based authentication. Patients can interact with the chatbot, manage their health profile, and book appointments. Doctors can manage patient data and generate AI-assisted SOAP notes. Admins can monitor system activity and manage users.

Key features include bilingual communication (English and Telugu), voice interaction, emergency keyword detection, and location-based hospital recommendations using OpenStreetMap APIs.

This project aims to enhance accessibility to healthcare services, provide immediate assistance during emergencies, and demonstrate the practical implementation of AI in real-world healthcare applications.

KEYWORDS: AI, NLP, Machine Learning, Spring Boot, React, etc.

TABLE OF CONTENTS

Abstract	I
Table of contents	II
List of figures	IV
List of tables	V
Abbreviations	VI

Chapter No.	Description	Page No.
1	INTRODUCTION	1-4
	1.1 Problem Statement	2
	1.2 Problem Overview	2
	1.3 Methodology	4
2	LITERATURE SURVEY	5-6
	2.1 Related Work	5
3	EXISTING SYSTEM AND PROPOSED METHOD	7-10
	3.1 Existing Method	7
	3.1.1 Limitations	7
	3.2 Proposed Method	8
	3.2.1 Block Diagram	9
4	HARDWARE AND SOFTWARE REQUIREMENTS	11-15

	4.1 Hardware Requirements.	11
	4.2 Software Requirements	12
	4.3 Functional and Non-Functional Requirements	12-
	4.3.1 Functional Requirements	12
	4.3.2 Non-Functional Requirements	14
5	SYSTEM DESIGN	16-25
	5.1 Architecture	16
	5.2 UML diagrams	18
6	IMPLEMENTATION	26-51
	6.1 Description of Modules	26
	6.2 Sample Source Code	27
7	TESTING	52-54
	7.1 Testing Techniques	52
	7.2 Test Cases	53
8	RESULTS	55-64
9	CONCLUTION AND FUTURE SCOPE	65-67
10	REFERENCES	68

LIST OF FIGURES

Fig No.	Name	Page No.
1	Proposed Healthcare Chatbot System	9
2	System Architecture of AI Healthcare Chatbot	16
3	Use Case Diagram of AI Healthcare Chatbot System	19
4	Class Diagram of AI Healthcare Chatbot System	21
5	Sequence Diagram of AI Healthcare Chatbot System	23
6	Deployment Diagram of AI Healthcare Chatbot System	24
7	Activity Diagram of AI Healthcare Chatbot System	25

LIST OF TABLES

Tab.no.	Name	Page No.
2.1	Comparative Analysis of Existing Systems	6
7.1	Test Cases for AI Healthcare Chatbot System	21

LIST OF ABBREVIATIONS

S.No	Abbreviation	Full Form
1	AI	Artificial Intelligence
2	NLP	Natural Language Processing
3	ML	Machine Learning
4	API	Application Programming Interface
5	JWT	JSON Web Token
6	LLM	Large Language Model
7	UI	User Interface
8	HTML	HyperText Markup Language
9	CSS	Cascading Style Sheets
10	SQL	Structured Query Language
11	ORM	Object-Relational Mapping
12	RBAC	Role-Based Access Control
13	SPA	Single Page Application
14	REST	Representational State Transfer
15	UML	Unified Modelling Language
16	TLS	Transport Layer Security
17	CSRF	Cross-Site Request Forgery
18	WHO	World Health Organization
19	EHR	Electronic Health Record

CHAPTER-1

INTRODUCTION

The healthcare sector has witnessed significant advancements with the integration of modern technologies, particularly Artificial Intelligence (AI). These advancements have enabled the development of intelligent systems that can assist users in accessing healthcare services more efficiently. In today's fast-paced world, there is an increasing demand for systems that can provide immediate medical guidance, reduce dependency on physical consultations, and improve overall accessibility to healthcare services.

AI-powered chat assistants have emerged as an effective solution to address these needs. These systems are capable of simulating human-like conversations and providing real-time responses to user queries. By leveraging natural language processing and machine learning techniques, healthcare chatbots can assist users in understanding symptoms, offering basic medical advice, and guiding them toward appropriate healthcare resources.

The proposed system, AI-Powered Healthcare Chat Assistant, is designed as a full-stack web application that integrates AI capabilities with modern web technologies. The system utilizes Spring Boot for backend development and React for frontend implementation, ensuring a scalable and user-friendly platform. The integration of Ollama (gemma:2b) enables local AI-based response generation, while OpenAI GPT-4 acts as a fallback to maintain reliability.

The application is designed to support multiple user roles, including patients, doctors, and administrators, allowing efficient management and interaction within the system. It also incorporates features such as bilingual communication (English and Telugu), voice input and output, emergency keyword detection, and location-based healthcare services.

By combining these technologies and features, the system aims to provide a comprehensive, accessible, and intelligent healthcare solution that enhances user experience and supports timely decision-making.

1.1 PROBLEM STATEMENT:

Access to timely and reliable healthcare assistance remains a major challenge for many individuals, especially in rural and semi-urban areas. Patients often face delays in consulting healthcare professionals due to factors such as long waiting times, lack of nearby medical facilities, and limited awareness about available healthcare services. In many cases, individuals tend to ignore early symptoms or rely on unverified online information, which can lead to incorrect self-diagnosis and serious health risks.

Traditional healthcare systems primarily depend on face-to-face consultations, which may not always be feasible for minor health concerns or emergency situations. There is a lack of systems that can provide immediate guidance and assist users in understanding their health conditions in real time.

Existing digital healthcare solutions also suffer from several limitations. Many applications are restricted to basic information delivery and do not provide intelligent interaction or personalized responses. They lack features such as emergency detection, real-time communication, and integration with location-based services. Additionally, most systems support only English, making them less accessible to users who are more comfortable with regional languages.

Another significant issue is the absence of voice-based interaction, which limits usability for individuals who are not familiar with typing or digital interfaces. Furthermore, the inability to quickly identify nearby hospitals or doctors during emergencies can result in delayed medical attention.

Therefore, there is a need for an intelligent and integrated system that can provide instant, reliable, and accessible healthcare assistance, while also supporting multilingual communication, emergency alerts, and real-time services.

1.2 PROJECT OVERVIEW

The AI-Powered Healthcare Chat Assistant is designed as a comprehensive web-based application that provides intelligent healthcare support through an interactive chatbot interface. The system integrates modern web technologies with AI capabilities to deliver real-time assistance and improve accessibility to healthcare services.

The application is developed using a full-stack architecture, where the backend is implemented using Spring Boot, handling business logic, authentication, and database operations. The frontend is built using React with Tailwind CSS, providing a responsive and user-friendly interface. Communication between frontend and backend is achieved through RESTful APIs.

A key component of the system is the AI integration. The application uses Ollama (gemma:2b) as the primary AI model to generate responses locally, ensuring faster processing and improved data privacy. In cases where the local model is unable to provide accurate responses, OpenAI GPT-4 is used as a fallback mechanism to maintain reliability.

The system supports multiple user roles, including Patient, Doctor, and Admin, each with specific functionalities. Patients can interact with the chatbot, manage their health profiles, and book appointments. Doctors can view patient information, manage appointments, and generate AI-assisted SOAP notes. Administrators can monitor system activities, manage users, and analyze performance through dashboards.

The application also includes advanced features such as bilingual communication (English and Telugu), enabling better accessibility for users. Voice-based interaction is implemented using the Web Speech API, allowing users to communicate with the system through speech. Additionally, the system includes an emergency detection feature that identifies critical keywords and provides immediate alerts along with emergency contact information.

Another important feature is the integration of location-based services using OpenStreetMap APIs. This allows users to find nearby hospitals and healthcare providers based on their current location, improving access to medical services during emergencies. Overall, the system provides a unified platform that combines AI, web technologies, and real-time services to deliver an efficient and user-friendly healthcare solution.

1.3 METHODOLOGY

The development of the AI-Powered Healthcare Chat Assistant follows a structured and systematic approach to ensure efficient implementation and high-quality performance.

The methodology involves multiple stages, each focusing on a specific aspect of the system development process.

The first stage is requirement analysis, where the needs and expectations of the system are identified. This includes defining the functionalities required for different user roles such as patients, doctors, and administrators. Both functional and non-functional requirements are analyzed to ensure the system meets user expectations and performance standards.

The next stage is system design, where the architecture of the application is planned. The system is divided into frontend, backend, and AI components, each designed to function independently while maintaining seamless integration. The database schema is designed using MySQL, and RESTful APIs are defined for communication between components.

In the implementation phase, the backend is developed using Spring Boot, which handles authentication, business logic, and data management. The frontend is implemented using React, providing a dynamic and interactive user interface. AI integration is achieved using Ollama for local response generation, with GPT-based APIs serving as a fallback.

Additional features such as voice interaction, emergency detection, and location services are integrated using appropriate APIs and technologies. These features enhance the functionality and usability of the system.

Once development is complete, the system undergoes testing, including unit testing, integration testing, and user interface testing, to ensure reliability and performance. Any identified issues are resolved to improve system stability.

Finally, the system is deployed locally, with configured ports for backend, frontend, and AI services. This ensures smooth communication between all components and enables the system to function effectively in a real-world environment.

CHAPTER-2

LITERATURE SURVEY

2.1 RELATED WORK:

Several AI-based healthcare systems and chatbot applications have been developed to provide medical assistance and improve patient interaction. Systems like Ada Health and Babylon Health use artificial intelligence to analyze user symptoms and provide possible health suggestions. Similarly, Woebot focuses on mental health support using conversational AI techniques.

Most existing systems utilize Natural Language Processing (NLP) to understand user queries and generate responses. Some platforms also offer features such as online consultations and appointment booking. Recent developments include the use of local AI models, such as Ollama, which improve data privacy and reduce dependency on cloud services.

However, many of these systems are limited in functionality, lacking features such as emergency detection, multilingual support, and integration with location-based healthcare services.

Many existing healthcare chatbot systems utilize Natural Language Processing (NLP) techniques to interpret user queries and provide relevant responses. These systems are designed to simulate human-like conversations and assist users in resolving their health-related doubts.

Platforms such as Ada Health and Babylon Health have demonstrated the effectiveness of AI in symptom analysis and virtual consultation, although they rely heavily on cloud-based infrastructure and subscription model.

Several research works and applications have focused on developing intelligent chatbot systems that assist users in understanding basic health information and connecting them with healthcare services.

S.No	Author/ System	Year	Description	Limitations
1	Ada Health	2019	AI symptom checker that analyzes user inputs and provides possible conditions	Limited to symptom checking, no emergency detection
2	Babylon Health	2020	AI chatbot with doctor consultation and video	support Subscription based and cloud dependent
3	Woebot	2018	Mental health chatbot using CBT techniques	Limited to mental health only
4	IBM Watson Health	2021	AI system for medical diagnosis using large datasets	High cost and complex implementation
5	HealthTap	2017	Connects users with doctors via chat	Not fully automated
6	Buoy Health	2020	AI assistant for symptom analysis using NLP	Lacks real-time services and integration
7	Ollama AI Systems	2023	Local AI models for healthcare chatbots	Lower accuracy than cloud models

Table 2.1: Comparative Analysis of Existing Systems

CHAPTER-3

EXISTING SYSTEM AND PROPOSED SYSTEM

3.1 EXISTING SYSTEM

The existing healthcare systems mainly focus on providing basic medical information and facilitating communication between patients and healthcare providers. Most systems are designed as standalone applications that perform limited functions such as symptom checking, appointment booking, or online consultation.

AI-based healthcare chatbots currently available in the market use predefined datasets and Natural Language Processing (NLP) techniques to respond to user queries. These systems allow users to input symptoms and receive general medical advice. However, many of these chatbots rely heavily on cloud-based APIs for processing user input, which can lead to increased latency and dependency on internet connectivity.

Traditional healthcare management systems are primarily used in hospitals and clinics to manage patient records, appointments, and billing information. While these systems are useful for administrative purposes, they do not provide real-time interaction or intelligent assistance to users.

Additionally, most existing systems are designed for a single type of user and do not support multi-role functionality. They also lack advanced features such as voice interaction, multilingual communication, and emergency detection, making them less effective in real-world scenarios.

3.1.1 LIMITATIONS

- The existing healthcare systems have several limitations that affect their performance and usability.

- One major limitation is the lack of intelligent and real-time interaction. Many systems provide only predefined responses and are unable to understand complex user queries effectively. This reduces the accuracy and usefulness of the system.
- Another limitation is the dependency on cloud-based services, which raises concerns related to data privacy, security, and cost. In addition, these systems may experience delays in response time due to network issues.
- Most existing applications do not support multiple user roles such as patients, doctors, and administrators. This limits their ability to provide a complete healthcare solution within a single platform.
- The absence of multilingual support is another drawback, as many systems are designed only for English-speaking users. This reduces accessibility for people who prefer regional languages.
- Furthermore, existing systems do not include emergency detection mechanisms, which are essential for identifying critical health conditions and providing immediate assistance. They also lack integration with location-based services, making it difficult for users to find nearby hospitals or healthcare providers.

3.2 PROPOSED SYSTEM:

The proposed system, AI-Powered Healthcare Chat Assistant, is designed to overcome the limitations of existing systems by providing a comprehensive and intelligent healthcare solution. The system integrates modern web technologies with AI capabilities to deliver real-time assistance to users.

The application is developed using a full-stack architecture, where the backend is implemented using Spring Boot and the frontend is built using React. The system uses JWT-based authentication to ensure secure access for different user roles, including patients, doctors, and administrators.

A key feature of the proposed system is the integration of AI models. The system uses Ollama (gemma:2b) as the primary model for generating responses locally, which improves

performance and ensures data privacy. In addition, OpenAI GPT-based APIs are used as a fallback mechanism to provide accurate responses when needed.

The system supports bilingual communication in English and Telugu, making it accessible to a wider range of users. It also includes voice input and output features, allowing users to interact with the system more conveniently.

An important feature of the proposed system is emergency detection. The system identifies critical keywords in user input and provides immediate alerts along with emergency contact information. Additionally, the integration of OpenStreetMap APIs enables users to find nearby hospitals and healthcare providers based on their location.

Overall, the proposed system provides a unified platform that combines AI, web technologies, and real-time services to deliver an efficient and user-friendly healthcare solution.

3.2.1 BLOCK DIAGRAM

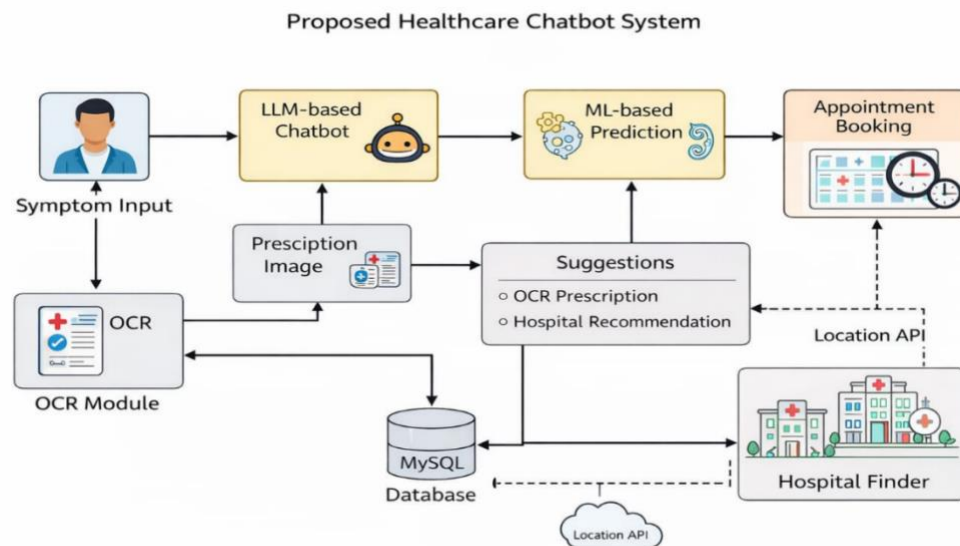


Fig. 3.1: Proposed Healthcare Chatbot System

- The block diagram of the proposed system represents the overall architecture and flow of data between different components of the application. The system consists of three main components: frontend, backend, and AI module.
- The frontend is developed using React and provides the user interface for interaction. Users can send queries, provide voice input, and access various features through the frontend.
- The backend, developed using Spring Boot, handles business logic, authentication, and database operations. It processes user requests, communicates with the AI module, and stores data in the MySQL database.
- The AI module, powered by Ollama, processes user input and generates responses. If required, the system uses GPT-based APIs as a fallback for better accuracy.
- The system also includes external services such as OpenStreetMap APIs for location-based features and Web Speech API for voice interaction. All components are interconnected to ensure smooth data flow and efficient system performance.

CHAPTER-4

HARDWARE AND SOFTWARE REQUIREMENTS

4.1 HARDWARE REQUIREMENTS

The hardware requirements define the physical components needed to develop and run the AI-Powered Healthcare Chat Assistant system efficiently.

Processor:

A minimum of Intel i5 or AMD Ryzen 5 processor is required to handle backend processing, frontend rendering, and AI operations. A higher processor improves overall system performance.

Memory(RAM):

At least 8 GB RAM is required for smooth multitasking. For better performance, especially when running AI models like Ollama, 16 GB RAM is recommended.

Storage:

A minimum of 20 GB free disk space is required to store application files, database, and AI models. SSD storage is preferred for faster data access.

Internet-Connectivity:

A stable internet connection is necessary for accessing external APIs such as OpenAI and OpenStreetMap, and for downloading dependencies.

GPU(Optional):

A dedicated GPU is not mandatory but can be used to accelerate AI processing and improve response time.

4.2 SOFTWARE REQUIREMENTS

The software requirements specify the tools, technologies, and platforms needed to develop and run the AI-Powered Healthcare Chat Assistant system.

- **OperatingSystem:**
Windows, Linux, or macOS for development and deployment.
- **BackendTechnologies:**
Java 17 and Spring Boot 3.2 are used to develop the backend services and APIs.
Spring Security is used for authentication and authorization.
- **FrontendTechnologies:**
React 18 (with Vite) is used to build the user interface, and Tailwind CSS is used for styling and responsive design.
- **Database:**
MySQL 8 is used to store user data, chat history, and appointment details.
- **AITools:**
Ollama (gemma:2b) is used as the primary AI model for generating responses locally. OpenAI GPT-4 is used as a fallback for reliability.
- **DevelopmentTools:**
Node.js and Maven are used for building and running frontend and backend applications. Git is used for version control.
- **Browser:**
Modern web browsers such as Google Chrome, Microsoft Edge, or Firefox are required to access the application.

4.3 FUNCTIONAL AND NON FUNCTIONAL REQUIREMENTS:

4.3.1 FUNCTIONAL REQUIREMENTS

User Authentication

- The system should allow users to register and login securely.

- It should support role-based access (Patient, Doctor, Admin).

AI Chat System

- Users should interact with an AI chatbot.
- The system should store chat history and support multiple sessions.
- Responses should be in English and Telugu.

Emergency Detection

- The system should detect emergency keywords (like chest pain).
- It should alert users with emergency messages and helpline numbers.

Voice Support

- The system should support voice input and text-to-speech output.

Location Services

- The system should fetch nearby hospitals/doctors using user location.
- If location is unavailable, use a default location.

Patient Features

- Manage health profile
- Book and view appointments
- View chat history

Doctor Features

- View patients
- Manage appointments
- Generate AI-based notes

Admin Features

- Manage users
- Monitor system and chat activity

- View analytics

Security

- Use JWT authentication
- Ensure data security and role-based access

AI Integration

- Use Ollama (primary) and OpenAI (fallback) for responses

4.3.2 NON-FUNCTIONAL REQUIREMENTS:

Performance

The system should respond quickly to user requests.

AI responses should be generated with minimal delay.

Security

The system should ensure secure authentication using JWT.

User data should be encrypted and protected.

Scalability

The system should handle multiple users simultaneously.

It should support future expansion (more users, features).

Usability

The interface should be user-friendly and easy to navigate.

It should support dark theme UI for better experience.

Reliability

The system should work continuously without failure.

It should provide fallback (OpenAI) if primary AI fails.

Availability

The system should be accessible 24/7.

Services should remain available even during high usage.

Maintainability

The code should be modular and easy to update.

Bugs and updates should be handled efficiently.

Accuracy

AI responses should be relevant and accurate.

Emergency detection should be reliable.

Compatibility

The system should work on different browsers and devices.

CHAPTER- 5

SYSTEM DESIGN

5.1 ARCHITECTURE:

The system architecture of the AI-Powered Healthcare Chat Assistant is designed using a **client-server model**, where different components work together to provide efficient, secure, and intelligent healthcare services. The architecture mainly consists of Frontend, Backend, Database, AIServices, External APIs.

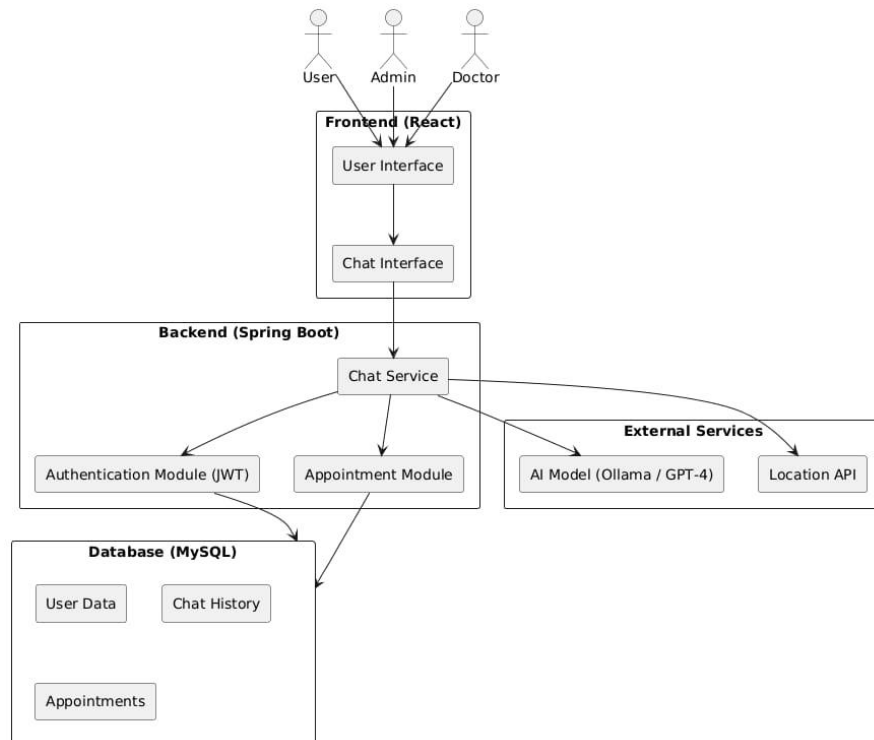


Fig. 5.1: System Architecture of AI-Powered Healthcare Chatbot

Architectural Flow Description:

1. Frontend Layer

The frontend is developed using React and provides an interactive user interface for patients, doctors, and admins. It allows users to perform actions such as login, chatting

with AI, booking appointments, and viewing dashboards. The frontend also supports voice input and text-to-speech output for better user experience. All user requests are sent to the backend through API calls.

2. Backend Layer

The backend is built using Spring Boot and acts as the core of the system. It handles all business logic and processes user requests. It includes different modules such as:

- Authentication Module for secure login using JWT
- Chat Module for handling AI conversations
- Appointment Module for booking and managing appointments
- Emergency Detection Module for identifying critical situations

The backend also communicates with the database and AI services.

3. Database Layer

The system uses MySQL database to store all application data. This includes:

- User details and roles
- Chat sessions and messages
- Health profiles
- Appointment records

The database ensures data is stored securely and can be retrieved efficiently.

4. AI Integration Layer

The system integrates AI models to generate responses:

- Ollama (gemma:2b) is used as the primary model for local processing, ensuring faster response and data privacy.

- OpenAI GPT-4 is used as a fallback model to maintain reliability when the primary model fails.

The AI layer processes user queries and returns intelligent responses.

5. External Services Layer

The system uses external APIs to enhance functionality:

- OpenStreetMap API to find nearby hospitals and doctors
- Web Speech API for voice recognition and speech output

These services help in providing location-based and voice-enabled features.

6. Data Flow

1. User interacts with the frontend (text/voice).
2. Request is sent to the backend via API.
3. Backend processes the request and interacts with:
 - Database (for storing/retrieving data)
 - AI services (for generating responses)
4. AI response is returned to the backend and then sent to the frontend.
5. Results are displayed to the user.

5.2 UML DIAGRAMS:

1. Use Case Diagram

The Use Case Diagram represents the interaction between the user and the system. In this system, the primary actor is the **Patient**, who interacts with various functionalities provided by the application. The diagram shows different use cases such as login, sending messages, receiving responses, uploading reports, and booking appointments.

Initially, the patient logs into the system using valid credentials. After successful authentication, the patient can send a message or query to the system. The system processes the request and generates an appropriate response, which is then displayed to the patient.

The patient can also upload medical reports for further analysis. These reports may be used by the system or doctors to provide better responses or recommendations. After reviewing the response or diagnosis, the patient can proceed to book an appointment with a doctor.

The diagram clearly shows the relationship between the actor and the system functions using arrows. Each oval represents a specific functionality of the system. The flow of actions indicates how a user interacts step-by-step.

This diagram helps in understanding the **functional requirements** of the system and ensures that all user interactions are properly captured. It is mainly used during the requirement analysis phase of software development.

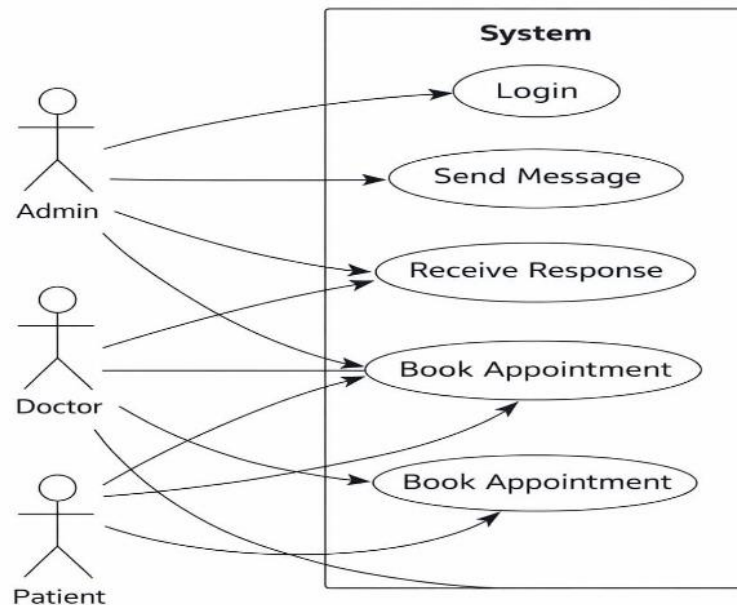


Fig.5.2: Use case Diagram

2. Class Diagram

The Class Diagram represents the static structure of the system by showing classes, attributes, methods, and relationships. It is one of the most important diagrams in object-oriented design.

In this system, the main classes include **User**, **Patient/User**, **Doctor**, and **ChatService/Chatbot**. The User class contains attributes such as `userId`, `name`, `email`, and `password`. It also includes methods like `login()` and `logout()`.

The Patient class extends the User class and includes additional functionality such as uploading reports. The Doctor class contains attributes like doctorId and specialization, representing medical professionals in the system.

The ChatService or Chatbot class is responsible for handling communication between the user and the system. It includes methods like sendMessage() and getResponse().

Relationships between classes are shown using arrows and inheritance symbols. These relationships define how classes interact and depend on each other.

The diagram helps developers understand the system structure and design better object-oriented code. It is mainly used during the design phase of software development.

Overall, the Class Diagram provides a blueprint for implementing the system in programming languages.

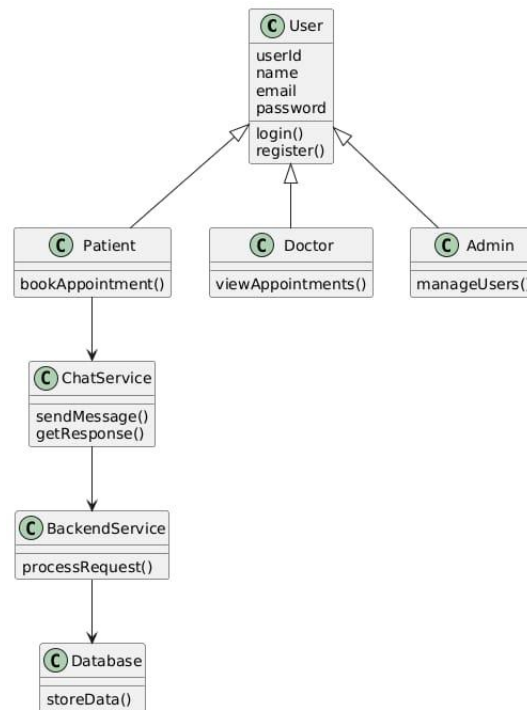


Fig.5.3: Class Diagram

3. Sequence Diagram

The Sequence Diagram shows how different components of the system interact with each other over time. It focuses on the order in which messages are exchanged between objects.

In this diagram, the main participants are **Patient, UI, Backend, and AI Processor**. The interaction begins when the patient sends a message through the user interface.

The UI forwards this request to the backend server. The backend processes the request and sends it to the AI processor for further analysis. The AI processor analyzes the query and generates a response.

The response is sent back to the backend, which then forwards it to the UI. Finally, the UI displays the response to the patient. The patient can then take further actions such as booking an appointment.

Each interaction is represented by arrows, and vertical lines represent the lifespan of each component. The diagram clearly shows the sequence of operations and communication flow.

This diagram is useful for understanding real-time interactions and debugging system processes. It helps developers design efficient communication between system components.

In some cases, the AI processor may request additional information to provide a more accurate response. The backend communicates this requirement to the UI, prompting the patient to input more details.

This iterative interaction continues until sufficient data is collected for accurate analysis. The sequence diagram can also represent optional flows, such as saving chat history or generating reports.

After delivering the response, the backend may store the interaction data in the database for future reference and analysis. This helps in improving system performance and maintaining user records

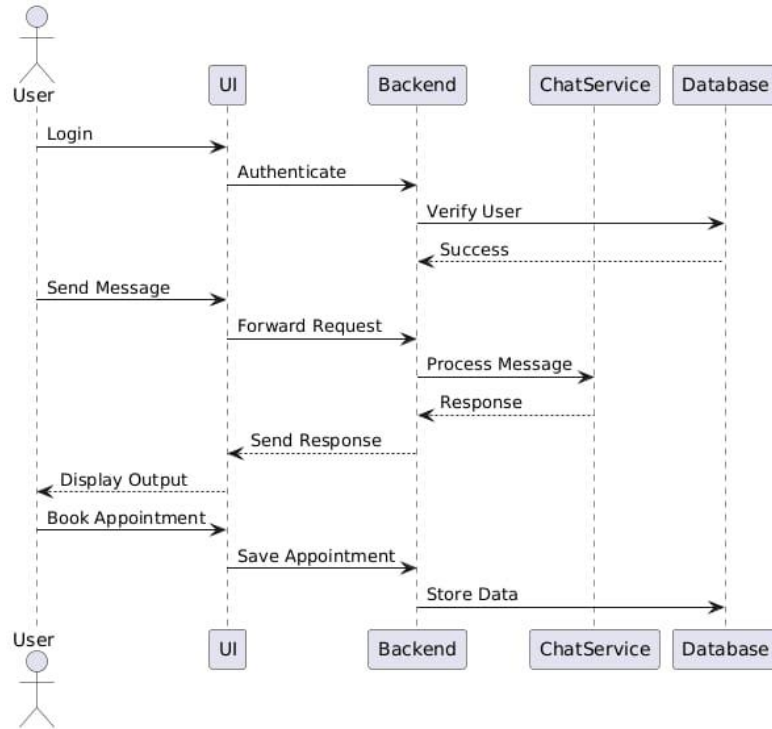


Fig.5.4: Sequence Diagram

4. Deployment Diagram

The Deployment Diagram illustrates the physical architecture of the system and how different components are deployed across various nodes. It shows how hardware and software components interact with each other in a real-time environment.

In this system, the **Client Device (Browser)** is used by the patient to access the application. The user interface is developed using a frontend framework such as React, which communicates with the backend server through HTTP requests.

The **Backend (Spring Boot API)** handles business logic and processes user requests. It acts as an intermediary between the frontend and the database. The backend also communicates with the AI server to process user queries.

The **AI Server (AI Processor)** is responsible for analyzing the input query and generating intelligent responses. It plays a key role in chatbot functionality.

The **Database (MySQL)** stores user data, reports, messages, and appointment details. The backend interacts with the database using queries to retrieve or store data.

The diagram shows communication using protocols like HTTP and API calls. It clearly defines the interaction between components and helps developers understand system deployment.

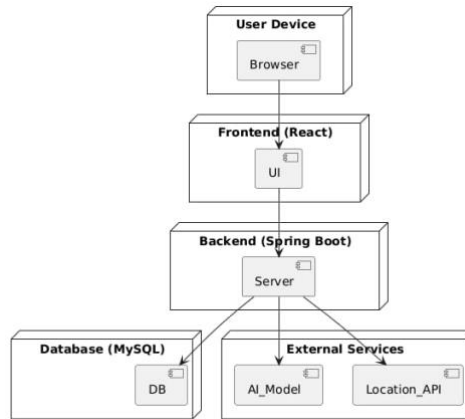


Fig.5.5: Deployment Diagram

5. Activity Diagram

The Activity Diagram represents the workflow of the system and shows how activities are performed step-by-step. It begins with an initial node and ends with the final output of the process.

The process starts when the user opens the application. After launching the app, the user enters a query related to their health issue. This query is then sent to the AI system for processing.

Once the query reaches the system, it is analyzed and processed using algorithms or AI models. After processing, the system generates an appropriate response based on the input.

The response is then displayed to the user. This entire process represents a continuous flow of activities. Each activity is represented by a rounded rectangle, and arrows indicate the direction of flow.

This diagram helps in understanding how the system behaves dynamically. It is useful for visualizing the sequence of operations and identifying possible improvements in workflow.

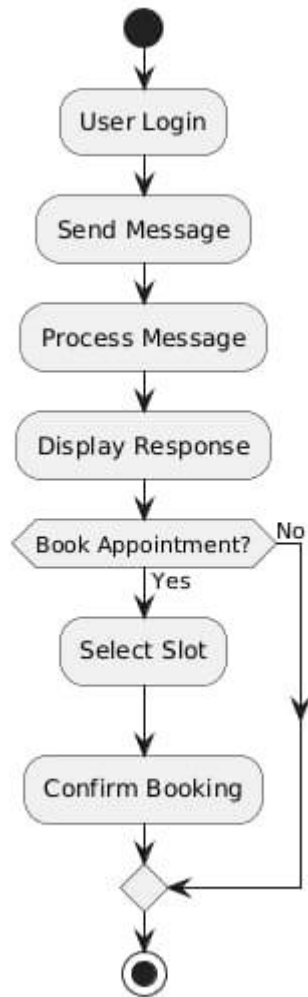


Fig.5.6: Activity Diagram

CHAPTER-6

IMPLEMENTATION AND RESULTS

6.1 Description of Modules

The AI-Powered Healthcare Chat Assistant is made up of parts. This makes the system easy to use and maintain. It also helps new features work together. The system has main parts:

1. Authentication and Security Module

This part is very important. It helps users join the system and makes sure they are who they say they are. It keeps users safe. Makes sure only the right people can get in. It uses codes called JSON Web Tokens to keep users safe. It also uses passwords and keeps them secret. There are three kinds of users:

- * Patient: Patients can only see their health information and talk to the chatbot.
- * Doctor: Doctors can see their patients information. Manage their schedules.
- * Admin: Admins can do everything. Manage the whole system.

2. AI (Chat) Module

This part is the main way users talk to the system. It uses computer programs to understand what users say and respond in a way that makes sense. It keeps track of what users say so it can respond better time. It also keeps users health information safe. All conversations are saved so the system can use them later.

3. Appointment Management Module

This part helps patients make appointments with doctors. Patients can make, change or cancel appointments easily. The system makes sure doctors do not get many appointments at the same time. It also sends reminders to patients so they do not forget their appointments.

4. Emergency Detection and Escalation Module

This part is very important. It helps users who need emergency help. It uses computer programs to detect if a user is in trouble. If it detects an emergency it sends an alert. Gives the user important instructions. It also shows the user where to get help.

5. Health Profile and EHR Module

This part keeps track of users health information. Users can. Update their health information easily. The system uses this information to give users advice.

6. Clinical Provider (Doctor) Module

This part is for doctors. It helps them manage their patients information and appointments. It also helps them write notes about their patients.

7. Administrative and Analytics Module

This part is for admins. It helps them manage the system and see how it is working. They can see how many users are, on the system and how well it is working.

8. Geolocation and Routing Services Module

This part helps users find help near them. It uses maps to show users where to go. It can also give users directions to get there.

9. Accessibility and Voice Support Module

This part helps users who need help. It allows users to talk to the system of typing. It also reads the systems responses loud. This makes it easier for users to use the system. The AI-Powered Healthcare Chat Assistant is made up of these modules.

6.2 SAMPLE SOURCE CODE

```
package com.healthcare;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.scheduling.annotation.EnableAsync;

/**
 * Main application class for Healthcare Chat Assistant
 */
@SpringBootApplication
@EnableAsync
public class HealthcareChatAssistantApplication {

    public static void main(String[] args) {
```

```
SpringApplication.run(HealthcareChatAssistantApplication.class,
args);
    }
}

package com.healthcare.controller;

import com.healthcare.dto.ApiResponse;
import com.healthcare.model.*;
import com.healthcare.repository.*;
import com.healthcare.service.AuthService;
import lombok.RequiredArgsConstructor;
import org.springframework.http.ResponseEntity;
import org.springframework.security.access.prepost.PreAuthorize;
import org.springframework.web.bind.annotation.*;

import java.time.LocalDateTime;
import java.time.format.DateTimeParseException;
import java.util.*;
import java.util.stream.Collectors;

/**
 * Controller for patient-specific operations
 */
@RestController
@RequestMapping("/api/patient")
@RequiredArgsConstructor
@CrossOrigin(origins = "*", maxAge = 3600)
public class PatientController {

    private final HealthProfileRepository healthProfileRepository;
    private final AppointmentRepository appointmentRepository;
    private final MedicalReportRepository medicalReportRepository;
    private final ChatSessionRepository chatSessionRepository;
    private final UserRepository userRepository;
```

```
private final RoleRepository roleRepository;
private final AuthService authService;

// _____ PROFILE
_____

@GetMapping("/profile")
@PreAuthorize("hasRole('PATIENT')")
public ResponseEntity<?> getHealthProfile() {
    try {
        User currentUser = authService.getCurrentUser();
        HealthProfile profile =
healthProfileRepository.findByUser(currentUser)
                            .orElseThrow(() -> new RuntimeException("Health
profile not found"));
        return ResponseEntity.ok(new ApiResponse(true, "Profile
retrieved", profile));
    } catch (Exception e) {
        return ResponseEntity.badRequest().body(new
ApiResponse(false, e.getMessage()));
    }
}

@PutMapping("/profile")
@PreAuthorize("hasRole('PATIENT')")
public ResponseEntity<?> updateHealthProfile(@RequestBody
HealthProfile updatedProfile) {
    try {
        User currentUser = authService.getCurrentUser();
        HealthProfile profile =
healthProfileRepository.findByUser(currentUser)
                            .orElseThrow(() -> new RuntimeException("Health
profile not found"));

        if (updatedProfile.getAge() != null) {
            profile.setAge(updatedProfile.getAge());
        }
    }
}
```

```
        if (updatedProfile.getWeight() != null) {
            profile.setWeight(updatedProfile.getWeight());
        }
        if (updatedProfile.getHeight() != null) {
            profile.setHeight(updatedProfile.getHeight());
        }
        if (updatedProfile.getBloodType() != null) {
            profile.setBloodType(updatedProfile.getBloodType());
        }
        if (updatedProfile.getAllergies() != null) {
            profile.setAllergies(updatedProfile.getAllergies());
        }
        if (updatedProfile.getChronicConditions() != null) {

profile.setChronicConditions(updatedProfile.getChronicConditions());
        }
        if (updatedProfile.getCurrentMedications() != null) {

profile.setCurrentMedications(updatedProfile.getCurrentMedications())
;
        }
        if (updatedProfile.getEmergencyContact() != null) {

profile.setEmergencyContact(updatedProfile.getEmergencyContact());
        }

        HealthProfile saved =
healthProfileRepository.save(profile);
        return ResponseEntity.ok(new ApiResponse(true, "Profile
updated", saved));
    } catch (Exception e) {
        return ResponseEntity.badRequest().body(new
ApiResponse(false, e.getMessage()));
    }
}
```



```
// _____ STATS

@GetMapping("/stats")
@PreAuthorize("hasRole('PATIENT')")
public ResponseEntity<?> getPatientStats() {
    try {
        User currentUser = authService.getCurrentUser();
        List<ChatSession> sessions =
chatSessionRepository.findByUserOrderByCreatedAtDesc(currentUser);
        List<MedicalReport> reports =
medicalReportRepository.findByUserOrderByUploadedAtDesc(currentUser);
        List<Appointment> appointments =
appointmentRepository.findByPatientOrderByAppointmentDateDesc(current
User);

        long upcoming = appointments.stream()
            .filter(a -> a.getStatus() ==
Appointment.AppointmentStatus.CONFIRMED
                || a.getStatus() ==
Appointment.AppointmentStatus.PENDING)
            .count();

        long emergencies = sessions.stream()
            .filter(s ->
Boolean.TRUE.equals(s.getIsEmergency()))
            .count();

        String riskLevel = emergencies > 0 ? "HIGH"
            : sessions.size() > 5 ? "MEDIUM"
            : "LOW";

        Map<String, Object> statsMap = new LinkedHashMap<>();
        statsMap.put("totalConsultations", sessions.size());
        statsMap.put("totalReports", reports.size());
        statsMap.put("upcomingAppointments", upcoming);
        statsMap.put("totalAppointments", appointments.size());
```

```
statsMap.put("emergencyAlerts", emergencies);
statsMap.put("riskLevel", riskLevel);

return ResponseEntity.ok(new ApiResponse(true, "Stats
retrieved", statsMap));
    } catch (Exception e) {
        return ResponseEntity.badRequest().body(new
ApiResponse(false, e.getMessage()));
    }
}

// ----- APPOINTMENTS
-----

@GetMapping("/appointments")
@PreAuthorize("hasRole('PATIENT')")
public ResponseEntity<?> getMyAppointments() {
    try {
        User currentUser = authService.getCurrentUser();
        List<Appointment> appointments =
appointmentRepository.findByPatientOrderByAppointmentDateDesc(current
User);

        return ResponseEntity.ok(new ApiResponse(true,
"Appointments retrieved", appointments));
    } catch (Exception e) {
        return ResponseEntity.badRequest().body(new
ApiResponse(false, e.getMessage()));
    }
}

@PostMapping("/appointments")
@PreAuthorize("hasRole('PATIENT')")
public ResponseEntity<?> bookAppointment(@RequestBody Map<String,
Object> body) {
    try {
        User currentUser = authService.getCurrentUser();
```

```
        Long doctorId =
Long.valueOf(body.get("doctorId").toString());
        User doctor = userRepository.findById(doctorId)
            .orElseThrow(() -> new RuntimeException("Doctor
not found"));

        String dateTimeStr =
body.get("appointmentDate").toString();
        LocalDateTime appointmentDate;
        try {
            appointmentDate = LocalDateTime.parse(dateTimeStr);
        } catch (DateTimeParseException e) {
            appointmentDate =
LocalDateTime.parse(dateTimeStr.replace(" ", "T"));
        }

        Appointment appt = new Appointment();
        appt.setPatient(currentUser);
        appt.setDoctor(doctor);
        appt.setAppointmentDate(appointmentDate);
        appt.setStatus(Appointment.AppointmentStatus.PENDING);
        if (body.get("reason") != null) {
            appt.setReason(body.get("reason").toString());
        }

        Appointment saved = appointmentRepository.save(appt);
        return ResponseEntity.ok(new ApiResponse(true,
"Appointment booked", saved));
    } catch (Exception e) {
        return ResponseEntity.badRequest().body(new
ApiResponse(false, e.getMessage()));
    }
}

@DeleteMapping("/appointments/{id}")
@PreAuthorize("hasRole('PATIENT')")
```

```
public ResponseEntity<?> cancelAppointment(@PathVariable Long id)
{
    try {
        User currentUser = authService.getCurrentUser();
        Appointment appt = appointmentRepository.findById(id)
            .orElseThrow(() -> new
RuntimeException("Appointment not found"));

        if
(!appt.getPatient().getId().equals(currentUser.getId())) {
            return ResponseEntity.status(403).body(new
ApiResponse(false, "Access denied"));
        }

        if (appt.getStatus() !=
Appointment.AppointmentStatus.PENDING) {
            return ResponseEntity.badRequest().body(new
ApiResponse(false, "Only PENDING appointments can be cancelled"));
        }

        appt.setStatus(Appointment.AppointmentStatus.CANCELLED);
        appointmentRepository.save(appt);
        return ResponseEntity.ok(new ApiResponse(true,
"Appointment cancelled", null));
    } catch (Exception e) {
        return ResponseEntity.badRequest().body(new
ApiResponse(false, e.getMessage()));
    }
}

// _____ DOCTORS
_____

@GetMapping("/doctors")
@PreAuthorize("hasRole('PATIENT')")
public ResponseEntity<?> getDoctors() {
    try {
```

```
        Role doctorRole =
roleRepository.findByName(Role.RoleName.ROLE_DOCTOR)
                .orElseThrow(() -> new RuntimeException("Doctor
role not found"));

        List<User> doctors =
userRepository.findByRolesContaining(doctorRole);
        // Return only safe fields
        List<Map<String, Object>> result = doctors.stream().map(d
-> {

                Map<String, Object> m = new LinkedHashMap<>();
                m.put("id", d.getId());
                m.put("username", d.getUsername());
                m.put("fullName", d.getFullName());
                m.put("email", d.getEmail());
                m.put("roles", d.getRoles().stream()
                        .map(r ->
r.getName().name()).collect(Collectors.toList()));
                return m;
        }).collect(Collectors.toList());
        return ResponseEntity.ok(new ApiResponse(true, "Doctors
retrieved", result));
    } catch (Exception e) {
        return ResponseEntity.badRequest().body(new
ApiResponse(false, e.getMessage()));
    }
}

// _____ EMERGENCY

@PostMapping("/emergency")
@PreAuthorize("hasRole('PATIENT')")
public ResponseEntity<?> sendEmergencyAlert() {
    try {
        User currentUser = authService.getCurrentUser();
        // Log emergency as an emergency chat session marker
        Map<String, Object> result = new LinkedHashMap<>();
```

```
        result.put("userId", currentUser.getId());
        result.put("username", currentUser.getUsername());
        result.put("timestamp", LocalDateTime.now().toString());
        result.put("message", "Emergency alert received for
patient: " + currentUser.getFullName());
        // In a production system this would trigger
SMS/email/push notifications
        return ResponseEntity.ok(new ApiResponse(true, "Emergency
alert sent successfully", result));
    } catch (Exception e) {
        return ResponseEntity.badRequest().body(new
ApiResponse(false, e.getMessage()));
    }
}
```

Frontend

```
import React from 'react';
import { FaPaperPlane } from 'react-icons/fa';

const ChatInput = ({ value, onChange, onSend, isSending }) => {

    return (
        <div className="w-full bg-white border-t border-gray-200 px-4
py-3 flex items-center gap-3 sticky bottom-0 z-20">
            <input
                type="text"
                className="flex-1 rounded-full border border-gray-300 px-
4 py-2 focus:outline-none focus:ring-2 focus:ring-primary-500 bg-
gray-50 text-gray-900 shadow-sm"
                placeholder="Type your message..."
                value={value}
                onChange={onChange}
                onKeyDown={e => e.key === 'Enter' && onSend()}
                disabled={isSending}
            />
        </div>
    )
}
```

```
<button
  className="ml-2 px-4 py-2 rounded-full bg-primary-600
hover:bg-primary-700 text-white font-semibold shadow transition-
colors disabled:opacity-50"
  onClick={onSend}
  disabled={isSending || !value.trim()}
  title="Send"
>
  <FaPaperPlane size={18} />
</button>
</div>
);
};

export default ChatInput;
import React from 'react';
import ReactMarkdown from 'react-markdown';

const ChatMessage = ({ message }) => {
  const isUser = message.role === 'USER';
  const isEmergency = message.isEmergency;

  return (
    <div className={`message-bubble ${isUser ? 'message-user' :
'message-assistant'}`}>
      {/* Role Badge */}
      <div className="flex items-center gap-2 mb-2">
        <span className="text-xs font-semibold text-gray-600">
          {isUser ? 'You' : 'AI Assistant'}
        </span>
        {isEmergency && (
          <span className="text-xs bg-red-600 text-white px-2 py-
1 rounded-full">
             EMERGENCY
          </span>
        )}
      </div>
    </div>
  )}
}
```

```

</div>

{/* Message Content */}
{isEmergency ? (
  <div className="bg-red-50 border-2 border-red-500
rounded-lg p-4">
    <div className="prose prose-sm max-w-none">
      <ReactMarkdown>{message.content}</ReactMarkdown>
    </div>
  </div>
) : (
  <div className="prose prose-sm max-w-none">
    <ReactMarkdown>{message.content}</ReactMarkdown>
  </div>
)}

{/* Structured Response for AI messages */}
{!isUser && message.structuredResponse && !isEmergency && (
  <div className="mt-4 space-y-3 border-t border-gray-200
pt-4">
    {message.structuredResponse.symptomSummary && (
      <div>
        <h4 className="text-sm font-semibold text-gray-700
mb-1"><img alt="Clipboard icon" data-bbox="238 611 258 628" style="vertical-align: middle;"/> Symptom Summary:</h4>
        <p className="text-sm text-gray-600">{message.structuredResponse.symptomSummary}</p>
      </div>
    )}

    {message.structuredResponse.possibleCauses && (
      <div>
        <h4 className="text-sm font-semibold text-gray-700
mb-1"><img alt="Magnifying glass icon" data-bbox="238 818 258 835" style="vertical-align: middle;"/> Possible Causes:</h4>
        <p className="text-sm text-gray-600">{message.structuredResponse.possibleCauses}</p>
      </div>
    )}
  </div>
)}

```



```

    })

    {message.structuredResponse.severityLevel && (
        <div>
            <h4 className="text-sm font-semibold text-gray-700
mb-1">⚠ Severity Level:</h4>
            <p className={`text-sm font-medium ${
                message.structuredResponse.severityLevel.toLowerCase().includes('high')
                    ? 'text-red-600'
                    :
message.structuredResponse.severityLevel.toLowerCase().includes('mode
rate')
                    ? 'text-yellow-600'
                    : 'text-green-600'
            }`}>
                {message.structuredResponse.severityLevel}
            </p>
        </div>
    })

    {message.structuredResponse.recommendedAction && (
        <div>
            <h4 className="text-sm font-semibold text-gray-700
mb-1">💡 Recommended Action:</h4>
            <p className="text-sm text-gray-
600">{message.structuredResponse.recommendedAction}</p>
        </div>
    })

    {message.structuredResponse.disclaimer && (
        <div className="bg-yellow-50 border border-yellow-200
rounded p-3">
            <p className="text-xs text-yellow-800">
                <strong>⚠ Disclaimer:</strong>
                {message.structuredResponse.disclaimer}
            </p>
        </div>
    })

```

```
        </p>
      </div>
    )}
  </div>
)}

  { /* Timestamp */}
  <div className="mt-3 text-xs text-gray-400">
    {new Date(message.createdAt).toLocaleString()}
  </div>
</div>
);
};

export default ChatMessage;

import React from 'react';
import ChatMessage from './ChatMessage';

const ChatWindow = ({ messages, isTyping }) => {
  return (
    <div className="flex flex-col h-full overflow-hidden">
      { /* Header */}
      <div className="sticky top-0 z-10 bg-white border-b border-
gray-200 shadow-sm">
        <div className="flex flex-col items-center py-4">
          <h1 className="text-2xl font-bold text-gray-900
tracking-tight">AI Healthcare Assistant</h1>
          <div className="mt-2 w-full flex justify-center">
            <span className="bg-yellow-100 text-yellow-800 text-
xs px-4 py-1 rounded-full font-medium border border-yellow-300">
              ⚠ Medical Disclaimer: This assistant does not
provide medical advice. Always consult a healthcare professional.
            </span>
          </div>
        </div>
      </div>
    </div>
  )
}
```

```

    </div>
    { /* Messages */
    <div className="flex-1 overflow-y-auto px-4 py-6 flex flex-
col gap-4 items-center bg-gray-50">
      {messages.map((msg, idx) => (
        <div key={idx} className="w-full flex justify-center">
          <div className="max-w-[70%] w-full">
            <ChatMessage message={msg} />
          </div>
        </div>
      ))}
      {isTyping && (
        <div className="w-full flex justify-start">
          <div className="max-w-[70%] w-full">
            <div className="flex items-center gap-2">
              <span className="inline-block w-2 h-2 bg-blue-400
rounded-full animate-bounce"></span>
              <span className="inline-block w-2 h-2 bg-blue-400
rounded-full animate-bounce delay-150"></span>
              <span className="inline-block w-2 h-2 bg-blue-400
rounded-full animate-bounce delay-300"></span>
              <span className="text-xs text-gray-400 ml-2">AI
is typing...</span>
            </div>
          </div>
        </div>
      )}
    </div>
  </div>
);
};

export default ChatWindow;
import React from 'react';
import { Link, useNavigate } from 'react-router-dom';
import { useAuth } from '../context/useAuth';

```

```
import { FaHeartbeat, FaUser, FaSignOutAlt, FaComments } from
'react-icons/fa';

const Navbar = () => {
  const { user, logout } = useAuth();
  const navigate = useNavigate();

  const handleLogout = () => {
    logout();
    navigate('/');
  };

  return (
    <nav className="bg-white shadow-md px-6 py-4">
      <div className="flex items-center justify-between">
        <div className="flex items-center gap-3">
          <FaHeartbeat className="text-3xl text-primary-600" />
          <h1 className="text-xl font-bold text-gray-
800">Healthcare Assistant</h1>
        </div>

        <div className="flex items-center gap-6">
          {user?.roles?.includes('ROLE_PATIENT') && (
            <>
              <Link
                to="/patient/chat"
                className="flex items-center gap-2 text-gray-700
hover:text-primary-600 transition-colors"
              >
                <FaComments />
                <span>Chat</span>
              </Link>
              <Link
                to="/patient/profile"
                className="flex items-center gap-2 text-gray-700
hover:text-primary-600 transition-colors"
            </>
          )}
        </div>
      </div>
    </nav>
  );
};
```

```
        >
        <FaUser />
        <span>Profile</span>
      </Link>
    </>
  )}

  <div className="flex items-center gap-3">
    <div className="text-right">
      <p className="text-sm font-medium text-gray-800">{user?.username}</p>
      <p className="text-xs text-gray-500">
        {user?.roles?.join(', ').replace(/ROLE_/g, '')}
      </p>
    </div>
    <button
      onClick={handleLogout}
      className="text-gray-600 hover:text-red-600 transition-colors"
      title="Logout"
    >
      <FaSignOutAlt size={20} />
    </button>
  </div>
</div>
</div>
</nav>
);
};

export default Navbar;

  <div className="flex-1 overflow-y-auto px-4 py-6 flex flex-col gap-4 items-center bg-gray-50">
    {messages.map((msg, idx) => (
      <div key={idx} className="w-full flex justify-center">
        <div className="max-w-[70%] w-full">
```

```

        <ChatMessage message={msg} />
      </div>
    </div>
  )})
  {isTyping && (
    <div className="w-full flex justify-start">
      <div className="max-w-[70%] w-full">
        <div className="flex items-center gap-2">
          <span className="inline-block w-2 h-2 bg-blue-400
rounded-full animate-bounce"></span>
          <span className="inline-block w-2 h-2 bg-blue-400
rounded-full animate-bounce delay-150"></span>
          <span className="inline-block w-2 h-2 bg-blue-400
rounded-full animate-bounce delay-300"></span>
          <span className="text-xs text-gray-400 ml-2">AI
is typing...</span>
        </div>
      </div>
    </div>
  )}
</div>

<div className="flex-1 overflow-y-auto px-4 py-6 flex flex-
col gap-4 items-center bg-gray-50">
  {messages.map((msg, idx) => (
    <div key={idx} className="w-full flex justify-center">
      <div className="max-w-[70%] w-full">
        <ChatMessage message={msg} />
      </div>
    </div>
  )})
  {isTyping && (
    <div className="w-full flex justify-start">
      <div className="max-w-[70%] w-full">
        <div className="flex items-center gap-2">
          <span className="inline-block w-2 h-2 bg-blue-400
rounded-full animate-bounce"></span>

```

```
        <span className="inline-block w-2 h-2 bg-blue-400
rounded-full animate-bounce delay-150"></span>
        <span className="inline-block w-2 h-2 bg-blue-400
rounded-full animate-bounce delay-300"></span>
        <span className="text-xs text-gray-400 ml-2">AI
is typing...</span>
      </div>
    </div>
  </div>
)}
</div>

import React, { useState, useMemo } from 'react';
import { adminAPI } from '../../services/api';
import {
  FaSearch, FaTrash, FaToggleOn, FaToggleOff,
  FaUserMd, FaUserInjured, FaShieldAlt, FaChevronLeft,
  FaChevronRight, FaUserPlus, FaTimes, FaCheck,
} from 'react-icons/fa';

const ROLE_COLORS = {
  ROLE_ADMIN: { label: 'Admin', bg: 'bg-red-100', text:
'text-red-700', icon: FaShieldAlt },
  ROLE_DOCTOR: { label: 'Doctor', bg: 'bg-violet-100', text:
'text-violet-700', icon: FaUserMd },
  ROLE_PATIENT: { label: 'Patient', bg: 'bg-blue-100', text:
'text-blue-700', icon: FaUserInjured },
};

const PAGE_SIZE = 10;

// Normalize role: could be a string "ROLE_ADMIN" or an object
{id, name}

const getRoleName = (role) => (typeof role === 'string' ? role :
role?.name);

const RoleBadge = ({ roles = [] }) => {
```

```
const primary = getRoleName(roles[0]);
if (!primary) return <span className="text-gray-300"></span>;
const cfg = ROLE_COLORS[primary] || { label: primary, bg: 'bg-gray-100', text: 'text-gray-700', icon: null };
const Icon = cfg.icon;
return (
  <span className={`inline-flex items-center gap-1.5 px-2.5 py-1 rounded-full text-xs font-medium ${cfg.bg} ${cfg.text}`}>
    {Icon && <Icon className="text-[10px]" />}
    {cfg.label}
  </span>
);
};

const UsersTable = ({ users, onRefresh }) => {
  const [search, setSearch] = useState('');
  const [roleFilter, setRoleFilter] = useState('ALL');
  const [page, setPage] = useState(1);
  const [confirmDelete, setConfirmDelete] = useState(null);
  const [actionLoading, setActionLoading] = useState(null);

  // Create Doctor modal
  const [showCreateDoctor, setShowCreateDoctor] =
useState(false);
  const [doctorForm, setDoctorForm] = useState({ username: '',
email: '', password: '', fullName: '', phoneNumber: '' });
  const [doctorMsg, setDoctorMsg] = useState(null);
  const [doctorLoading, setDoctorLoading] = useState(false);

  const filtered = useMemo(() => {
    let list = users;
    if (roleFilter !== 'ALL') {
      list = list.filter(u => u.roles?.some(r => getRoleName(r)
=== roleFilter));
    }
    const q = search.toLowerCase();
```



```
    if (q) {
      list = list.filter(u =>
        u.username?.toLowerCase().includes(q) ||
        u.email?.toLowerCase().includes(q) ||
        u.fullName?.toLowerCase().includes(q)
      );
    }
    return list;
  }, [users, search, roleFilter]);

  const totalPages = Math.max(1, Math.ceil(filtered.length /
    PAGE_SIZE));
  const currentPage = Math.min(page, totalPages);
  const pageData = filtered.slice((currentPage - 1) * PAGE_SIZE,
    currentPage * PAGE_SIZE);

  const handleToggle = async (userId) => {
    setActionLoading(userId + '-toggle');
    try {
      await adminAPI.toggleUserStatus(userId);
      onRefresh();
    } finally {
      setActionLoading(null);
    }
  };

  const handleDelete = async () => {
    if (!confirmDelete) return;
    setActionLoading(confirmDelete + '-delete');
    try {
      await adminAPI.deleteUser(confirmDelete);
      setConfirmDelete(null);
      onRefresh();
    } finally {
      setActionLoading(null);
    }
  }
```

```
};

const handleCreateDoctor = async (e) => {
  e.preventDefault();
  setDoctorLoading(true);
  setDoctorMsg(null);
  try {
    const res = await adminAPI.createDoctor(doctorForm);
    if (res.data.success) {
      setDoctorMsg({ type: 'success', text: 'Doctor account
created!' });
      setDoctorForm({ username: '', email: '', password: '',
fullName: '', phoneNumber: '' });
      onRefresh();
    }
  } catch (err) {
    setDoctorMsg({ type: 'error', text:
err.response?.data?.message || 'Failed to create doctor' });
  } finally {
    setDoctorLoading(false);
  }
};

return (
  <div className="space-y-5">
    { /* Toolbar */ }
    <div className="flex flex-wrap gap-3 items-center justify-
between">
      <div className="flex gap-3 flex-wrap">
        { /* Search */ }
        <div className="relative">
          <FaSearch className="absolute left-3 top-1/2 -
translate-y-1/2 text-gray-400 text-xs" />
          <input
            value={search}

```

```

        onChange={e => { setSearch(e.target.value);
setPage(1); }}
        placeholder="Search name, username, email..."
        className="pl-8 pr-3 py-2 text-sm border border-
gray-200 rounded-lg focus:outline-none focus:ring-2 focus:ring-blue-
300 w-64"
    />
</div>
{ /* Role filter */}
<select
    value={roleFilter}
    onChange={e => { setRoleFilter(e.target.value);
setPage(1); }}
    className="text-sm border border-gray-200 rounded-lg
px-3 py-2 focus:outline-none focus:ring-2 focus:ring-blue-300"
    >
    <option value="ALL">All Roles</option>
    <option value="ROLE_ADMIN">Admin</option>
    <option value="ROLE_DOCTOR">Doctor</option>
    <option value="ROLE_PATIENT">Patient</option>
    </select>
</div>
<button
    onClick={() => setShowCreateDoctor(true)}
    className="flex items-center gap-2 px-4 py-2 bg-blue-
600 text-white text-sm font-medium rounded-lg hover:bg-blue-700
transition-colors shadow-sm"
    >
    <FaUserPlus />
    Add Doctor
    </button>
</div>

{ /* Table */}
<div className="bg-white rounded-xl border border-gray-100
shadow-sm overflow-hidden">

```

```

<div className="overflow-x-auto">
  <table className="w-full text-sm">
    <thead>
      <tr className="border-b border-gray-100 bg-gray-
50/60">
        {['Name', 'Username', 'Email', 'Role', 'Status',
'Actions'].map(h => (
          <th key={h} className="px-5 py-3 text-left
text-xs font-semibold text-gray-500 uppercase tracking-wide
whitespace-nowrap">{h}</th>
        ))}
      </tr>
    </thead>
    <tbody className="divide-y divide-gray-50">
      {pageData.length === 0 ? (
        <tr>
          <td colspan={6} className="px-5 py-10 text-
center text-gray-400">No users found</td>
        </tr>
      ) : pageData.map(user => {
        const isActive      = user.isActive ??
user.active;
        const isDeleting    = actionLoading === user.id
+ '-delete';
        const isToggling    = actionLoading === user.id
+ '-toggle';
        return (
          <tr key={user.id} className="hover:bg-gray-
50/70 transition-colors">
            <td className="px-5 py-3.5 font-medium text-
gray-800">
              <div className="flex items-center gap-2.5">
                <div className="w-8 h-8 rounded-full bg-
gradient-to-br from-blue-400 to-blue-600 flex items-center justify-
center text-white text-xs font-bold flex-shrink-0">
                  <button onClick={() => setConfirmDelete(null)}

```

```
        className="flex-1 px-4 py-2 border border-gray-
200 rounded-xl text-sm font-medium hover:bg-gray-50 transition-
colors">
            Cancel
        </button>
        <button onClick={handleDelete}
            className="flex-1 px-4 py-2 bg-red-600 text-white
rounded-xl text-sm font-medium hover:bg-red-700 transition-colors
flex items-center justify-center gap-2">
            {actionLoading === confirmDelete + '-delete'
            ? <span className="w-4 h-4 border-2 border-
Create Doctor
            </button>
        </div>
    </form>
</div>
</div>
    )}
</div>
);
};

export default UsersTable;
```

CHAPTER-7

TESTING

7.1 TESTING TECHNIQUES

Testing techniques are used to ensure that the AI-Powered Healthcare Chat Assistant works correctly, efficiently, and securely.

1. Unit Testing

Individual components such as controllers, services, and methods are tested separately to verify that each part functions correctly.

2. Integration Testing

Different modules like frontend, backend, database, and AI services are tested together to ensure proper communication and data flow.

3. Functional Testing

All system functionalities such as login, chat, appointment booking, and emergency detection are tested to confirm they meet the requirements.

4. API Testing

Backend APIs are tested using tools like Postman to ensure correct request and response handling.

5. User Interface (UI) Testing

The frontend interface is tested to ensure it is user-friendly, responsive, and works properly on different devices and browsers.

6. Performance Testing

The system is tested for speed and responsiveness, especially for AI responses and multiple user requests.

7. Security Testing

Security features like JWT authentication and data protection are tested to prevent unauthorized access.

8. User Acceptance Testing (UAT)

The system is tested by users to ensure it meets real-world requirements and provides a satisfactory experience.

7.2 TEST CASES

The following test cases are designed to validate the functionality, performance, and reliability of the AI-Enabled Healthcare Chatbot system

Test Case Id	Test Case Name	Input	Expected Output	Actual Output	Status
TC-01	User Registration	Valid user details (name, email, password)	User registered successful	As Expected	Pass
TC-02	User Login	Valid username & password	Login successful, dashboard displayed	As Expected	Pass
TC-03	Invalid Login	Wrong credentials	Error message displayed	As Expected	Pass
TC-04	Symptom Input	User enters symptoms (fever, headache)	System asks follow-up questions	As Expected	Pass
TC-05	Disease Prediction	Symptoms Input	Predicted disease with confidence	As Expected	Pass

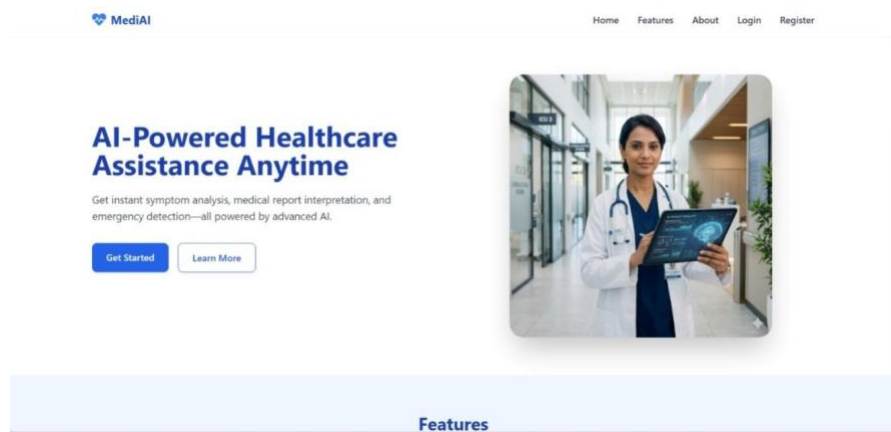
TC-06	Chatbot Response	User query message	Relevant AI response	As Expected	Pass
TC-07	OCR Processing	Upload prescription image	Text extracted using OCR	As Expected	Pass
TC-08	Hospital Search	User location	Nearby hospitals displayed	As Expected	Pass
TC-09	JWT Authentication	Valid token & time	Appointment confirmed	As Expected	Pass
TC-10	Role-Based Access	Admin/Doctor login	Access based on role	As Expected	Pass
TC-11	System Performance	Multiple users request	System responds within time	As Expected	Pass
TC-15	Error Handling	Invalid input / empty fields	Proper error message, no crash	As Expected	Pass

Table 7.1: Test Cases for AI HealthCare Chatbot System

Chapter -8

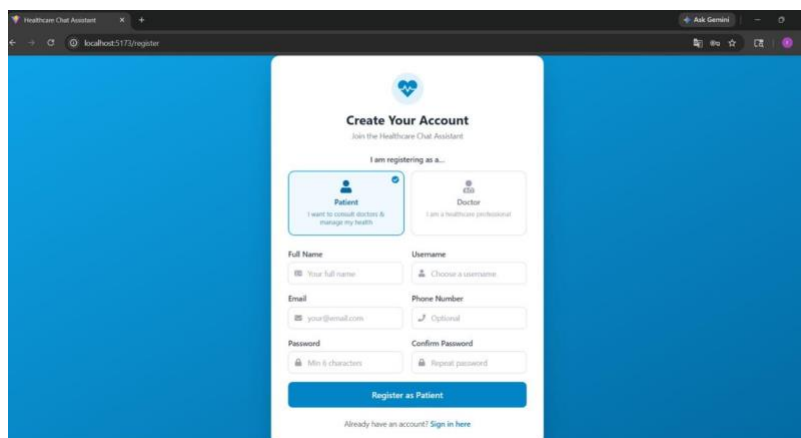
RESULT

Home Page



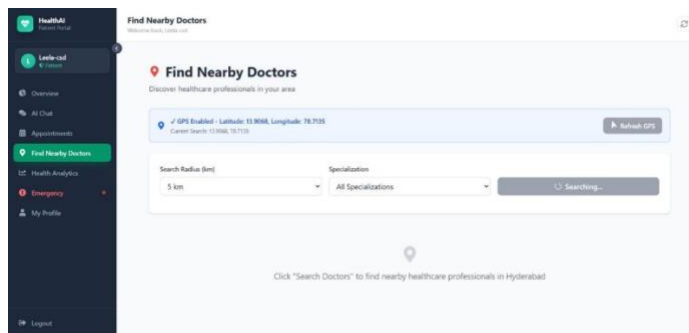
The Home Page introduces the AI-powered healthcare system to users. It highlights the main features such as symptom analysis, medical report interpretation, and emergency detection. The design is clean and attractive, making a good first impression. It includes buttons like “Get Started” and “Learn More” to guide users. The page clearly explains the purpose of the system. It shows how AI can improve healthcare services. The use of images and headings makes the content engaging. It is designed to attract users and encourage them to register. The layout is responsive and works on different devices. It builds trust by presenting a professional interface. The page acts as an entry point to the system. Overall, it effectively communicates the benefits of the application.

Registration Page

The screenshot shows the registration page of the MediAI application. The page has a blue background. At the top, there is a heading "Create Your Account" and a subheading "Join the Healthcare Chat Assistant". Below this, there is a section "I am registering as a..." with two options: "Patient" (selected) and "Doctor". The "Patient" option has a description: "I want to consult doctors & manage my health". The "Doctor" option has a description: "I am a healthcare professional". Below these options, there are input fields for "Full Name", "Email", "Phone Number", "Username", "Password", and "Confirm Password". There is a "Register as Patient" button at the bottom. At the very bottom, there is a link: "Already have an account? Sign in here".

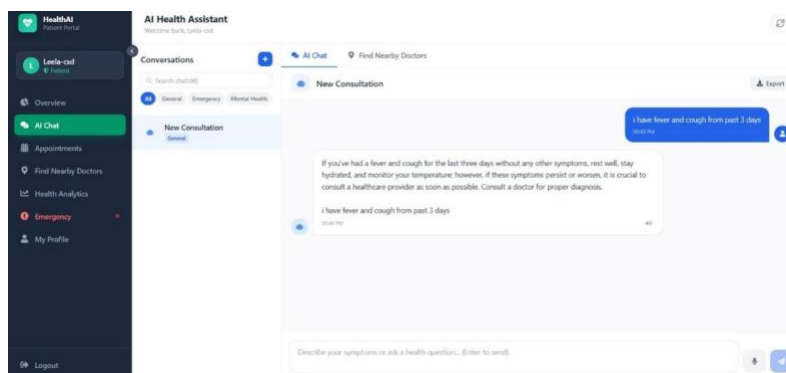
The Registration Page allows users to create an account easily. Users can register as a patient or doctor. The form collects details like name, email, phone number, and password. It ensures secure access to the system. The interface is simple and user-friendly. It guides users through the process step by step. Password confirmation ensures data accuracy. The system maintains privacy and security. It supports role-based access control. This means different users get different features. The design is clean and professional. It builds trust among users. Overall, it ensures safe and smooth onboarding.

Find Nearby Doctors



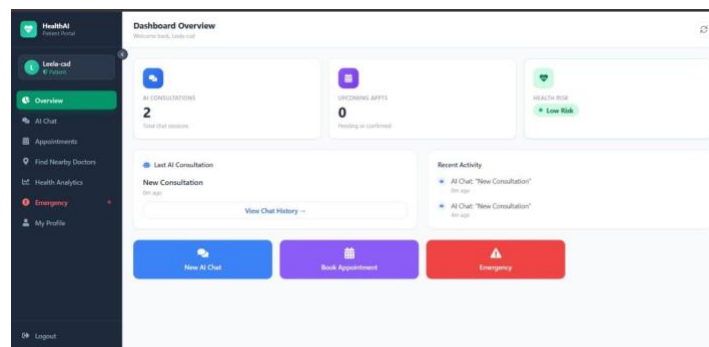
This feature helps users find doctors based on their current location. It uses GPS to detect latitude and longitude. Users can search within a specific radius. They can also filter doctors based on specialization. This makes it easier to find suitable healthcare professionals. The system provides accurate and quick results. It saves time and effort for patients. It improves accessibility to healthcare services. This feature is especially useful in urgent situations. It connects patients with nearby doctors quickly. Overall, it enhances convenience and efficiency.

AI Chat Assistant



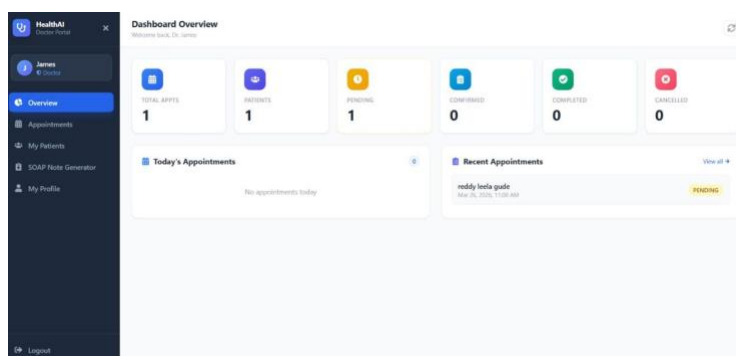
The AI Chat Assistant provides instant medical guidance to users. Patients can enter their symptoms and receive suggestions. The system analyzes input and gives basic advice. It helps users understand their condition. It also suggests when to consult a doctor. The chat interface is simple and interactive. Users can view chat history and previous consultations. It is available 24/7 for continuous support. This reduces dependency on immediate doctor visits. The feature improves accessibility to healthcare information. It saves time and provides quick assistance. Overall, it enhances user experience and supports better health management.

Patient Dashboard



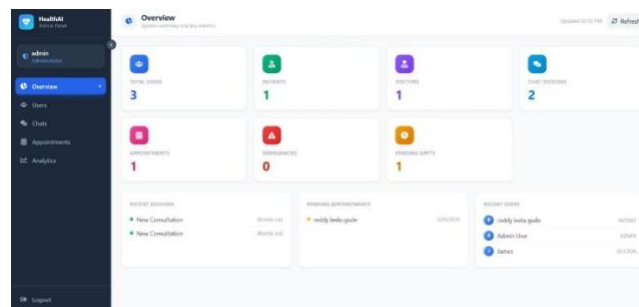
The Patient Dashboard provides all essential healthcare features in one place. It shows AI consultations, upcoming appointments, and health risk status. Patients can easily track their health activities. The dashboard also displays recent activity and chat history. This helps users stay updated with their interactions. It includes quick action buttons like “New AI Chat,” “Book Appointment,” and “Emergency.” These options improve accessibility. The interface is simple and easy to understand. Patients can navigate without any confusion. The dashboard reduces dependency on physical visits. Overall, it enhances user experience and convenience.

Doctor Dashboard



The Doctor Dashboard provides an overview of all activities related to patient care. It shows the total number of appointments, patients, pending requests, and completed consultations. The interface is designed to be clean and easy to use. Doctors can access patient details and track appointment status. It supports better communication between doctors and patients. The system reduces paperwork and manual scheduling. It allows doctors to focus more on patient care. The dashboard also provides real-time updates. This ensures that doctors are always informed about their tasks. Overall, it improves efficiency and enhances healthcare delivery.

Admin Dashboard – Overview



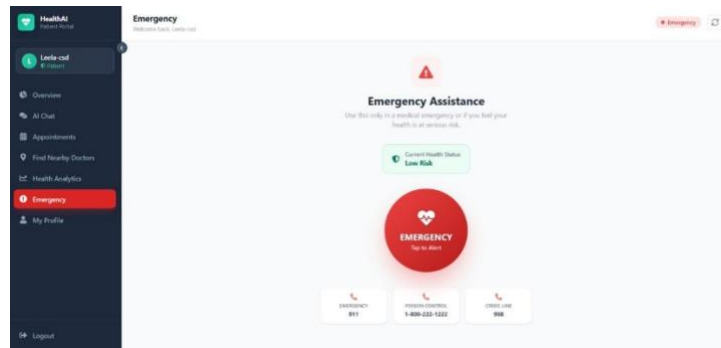
The Admin Dashboard provides a complete summary of the healthcare system in a single view. It displays key metrics such as total users, number of patients, doctors, chat sessions, and appointments. This helps the admin quickly understand the overall system activity. The dashboard also shows important sections like recent sessions, pending appointments, and recent users. These sections allow the admin to track ongoing activities in real time. The use of cards and visual indicators improves readability and decision-making. The dashboard acts as the control center of the entire application. It helps in managing healthcare services efficiently. The admin can easily identify system performance and user engagement. The layout supports quick navigation between

Appointment Booking Page

The screenshot shows the 'Book Appointment' modal form. It includes a 'Select Doctor' dropdown, a 'Date' field (26-03-2026), and a 'Time Slot' table. The table has columns for '09:00 AM', '10:00 AM', '11:00 AM', '12:00 PM', '01:00 PM', '02:00 PM', '03:00 PM', and '04:00 PM'. The '10:00 AM' slot is highlighted in green. Below the table, there is a 'Cancel' button and a 'Confirm Booking' button. A note at the bottom states: 'I have been suffering from headache from past 5 days'.

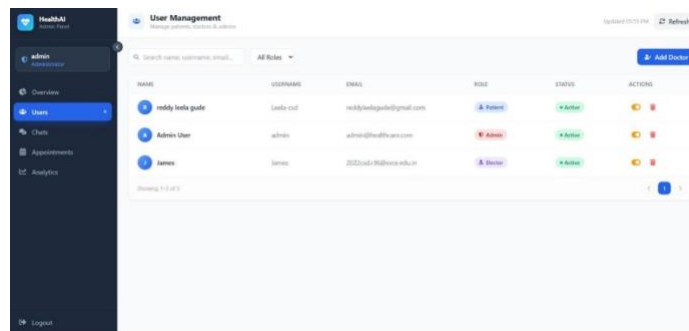
The Appointment Booking page allows patients to easily schedule consultations with doctors. Users can select the doctor, choose a date, and pick a suitable time slot. The system displays available time slots clearly, making it easy to choose. Patients can also add a reason for the appointment. It also prevents overlapping schedules. The confirmation button finalizes the booking instantly. Patients receive a clear view of their selected details. This improves transparency and trust. Overall, the feature enhances convenience for both patients and doctors.

Emergency Page



The Emergency Page is designed for critical situations where immediate help is required. It includes a large emergency button for quick access. Users can trigger alerts instantly. The page also shows the current health risk status. This helps users understand their condition. It provides emergency contact numbers for quick communication. The design is simple and focused on speed. It reduces delay during emergencies. The feature ensures user safety and quick response. It can be life-saving in serious conditions. The system prioritizes emergency handling. It improves reliability and trust in the application. Overall, it adds an important safety feature to the system.

Admin – User Management



The User Management module allows the admin to control all users in the system. It displays a list of patients, doctors, and admins. Each user's details such as name, email, and role are shown clearly. The admin can activate or deactivate users easily. It helps maintain system security. The delete option allows removal of unwanted users. The search and filter options make it easy to find specific users. The system supports efficient user handling. It reduces manual effort in managing accounts. The interface is clean and organized. It ensures proper control over system access. The module improves overall system administration. It also helps in monitoring user activity. Overall, it plays a key role in managing the healthcare platform effectively.

CHAPTER 9

CONCLUSION & FUTURE SCOPE

9.1 CONCLUSION

The AI-Enabled Healthcare Chatbot system developed in this project demonstrates the effective integration of Artificial Intelligence, web technologies, and real-time communication to provide accessible and user-friendly healthcare assistance. The system is designed to support users by offering essential healthcare services such as symptom-based guidance, prescription understanding through OCR, hospital recommendations, and appointment booking, without performing direct medical diagnosis.

The proposed system utilizes modern technologies including Java (Spring Boot) for backend development, React with Tailwind CSS for frontend design, and AI-based conversational models for intelligent interaction. The chatbot is capable of understanding user queries using Natural Language Processing (NLP) and responding in a meaningful and interactive manner. This enhances user experience and makes the system more practical for real-world applications.

One of the key highlights of the system is its ability to extract and interpret medical prescription details using Optical Character Recognition (OCR). This feature helps users understand their prescriptions more clearly, reducing confusion and improving medication awareness. Additionally, the system provides location-based hospital recommendations using mapping services, which is particularly useful during emergencies or for users in unfamiliar locations. The system also supports secure user authentication using JWT and provides role-based access for patients, doctors, and administrators. This ensures data security and proper management of system functionalities. The modular architecture of the system allows easy scalability and maintenance, making it suitable for future enhancements.

Unlike traditional systems, this project focuses on providing assistance and guidance rather than diagnosis, thereby maintaining ethical standards and avoiding the risks associated with incorrect medical predictions. The chatbot acts as a supportive tool that bridges the gap between users and healthcare services.

However, the system may face limitations such as dependency on user input accuracy, internet connectivity for certain services, and limited understanding of highly complex medical queries.

These challenges can be addressed with further improvements in AI models and system integration.

Overall, the AI-Enabled Healthcare Chatbot provides a reliable, efficient, and scalable solution for improving healthcare accessibility. It highlights the potential of combining artificial intelligence with web applications to create intelligent systems that enhance user experience and support everyday healthcare

9.2 FUTURE SCOPE

Although the proposed AI-Enabled Healthcare Chatbot system provides a robust and functional platform for healthcare assistance, there are several opportunities for future enhancements to improve its performance, usability, and scope.

One of the major improvements can be the integration of advanced Natural Language Processing models, which can better understand complex user queries and provide more accurate and context-aware responses. This will enhance the conversational capability of the chatbot and improve user satisfaction.

The system can also be extended by implementing voice-based interaction, allowing users to communicate with the chatbot through speech instead of text. This will be especially beneficial for elderly users and those who are not comfortable typing.

Another important enhancement is the development of a mobile application version of the system. A dedicated mobile app can provide better accessibility, push notifications, and real-time updates, making the system more convenient for daily use.

The OCR module can be further improved by integrating advanced image processing techniques to increase accuracy in extracting handwritten prescriptions. This will make the system more reliable in real-world scenarios where prescription clarity varies.

The chatbot can also be enhanced by adding multi-language support, enabling users to interact in regional languages such as Telugu, Hindi, and others. This will significantly increase the reach and usability of the system, especially in rural areas.

Integration with real-time healthcare APIs and hospital management systems can further improve the system by providing live appointment availability, doctor details, and emergency services. This will make the platform more dynamic and useful.

Another potential improvement is the inclusion of personalized user profiles, where the system can store user preferences, past interactions, and medical history (non-diagnostic), allowing the chatbot to provide more customized assistance.

The system can also incorporate notification and alert systems, such as reminders for medication, appointments, and follow-ups, improving user engagement and healthcare management.

Finally, the project can be extended to support wearable device integration and IoT-based health monitoring systems, enabling real-time tracking of basic health parameters and providing proactive healthcare suggestions (without diagnosis).

In conclusion, the future scope of this project lies in enhancing intelligence, accessibility, and integration capabilities. With continuous improvements, the AI Healthcare Chatbot can evolve into a comprehensive digital healthcare assistant that supports users efficiently while maintaining ethical boundaries.

CHAPTER-10

REFERENCES

- Ian Goodfellow, Yoshua Bengio, and Aaron Courville, Deep Learning, MIT Press, 2016.
- Stuart Russell and Peter Norvig, Artificial Intelligence: A Modern Approach, 3rd Edition, Pearson, 2010.
- Daniel Jurafsky and James H. Martin, Speech and Language Processing, Pearson, 2019.
- Tom Mitchell, Machine Learning, McGraw-Hill Education, 1997.
- Spring Boot Official Documentation. Available: <https://spring.io/projects/spring-boot>
- React Official Documentation. Available: <https://react.dev>
- Tailwind CSS Documentation. Available: <https://tailwindcss.com>
- MySQL Documentation. Available: <https://dev.mysql.com/doc>
- OpenAI API Documentation. Available: <https://platform.openai.com/docs>
- Ollama AI Documentation. Available: <https://ollama.com>
- Tesseract OCR Documentation. Available: <https://github.com/tesseract-ocr/tesseract>
- OpenStreetMap API Documentation. Available: <https://www.openstreetmap.org>
- JWT (JSON Web Token) Documentation. Available: <https://jwt.io>
- Research Paper: “AI Chatbots in Healthcare: Opportunities and Challenges,” IEEE Xplore, 2020.
- Research Paper: “Natural Language Processing for Healthcare Applications,” Springer, 2019

